

70-467 Machine Learning for Business Analytics

Model selection and regularization

Professor: Andrés Castaño Zuluaga

16 September 2026 · CMU-Q

Discussion 3

Canvas discussion: Humans still matter more than AI in finance

- The article says the 2025 “AI native” interns looked impressive until seniors asked follow-up questions. Why would a firm care about that? Think of an example applied to Business Analytics.

Pair presentation — model regularization (after we discuss model selection)

Ose and **Miron**

Topic: regularization

Then we continue the discussion of the methods.

Where are we on the course roadmap?

Last week we split, and we cross-validated, so we can make sure that predictions outside the sample are useful.

In this class we will discuss over the idea of how to select the best predictors for a given model.

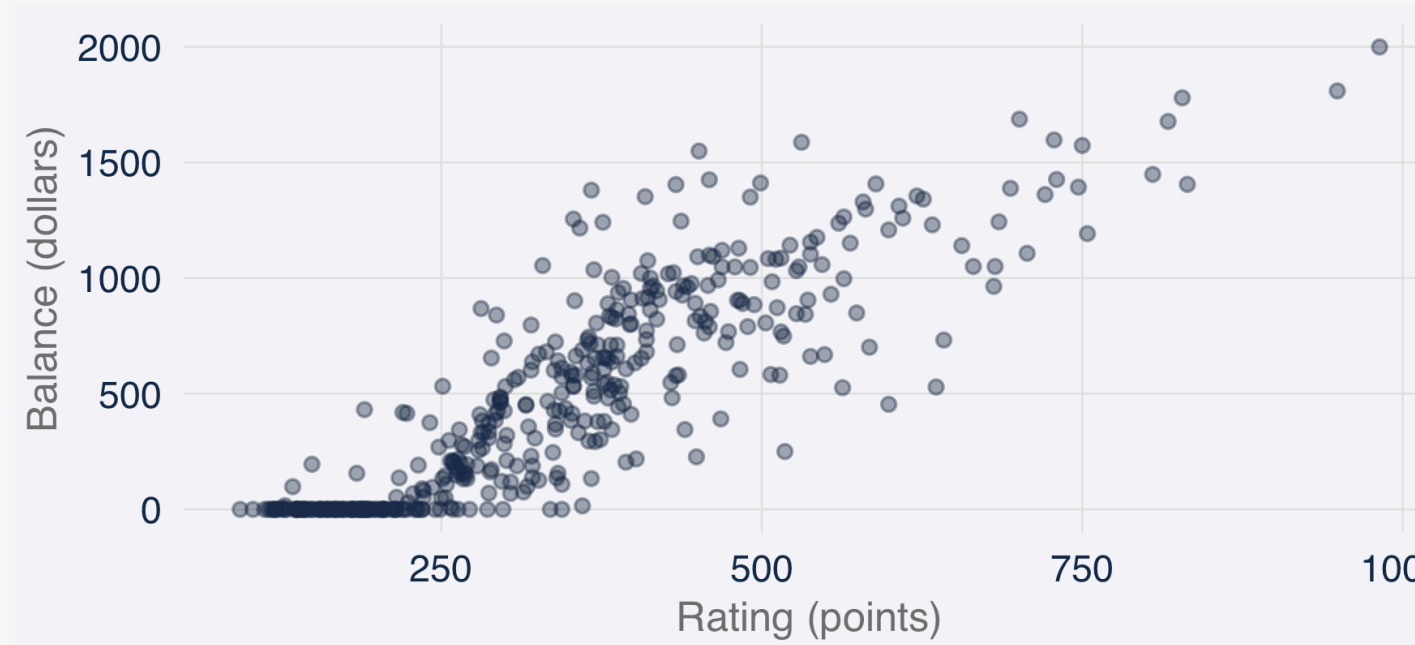
Topic	Description	Tools
FIT & EVALUATE		
Linear regression	Predict a number	OLS
Metrics	Size of the errors	RMSE, R ²
Train/test, leakage, cross-validation	Error on new data	Holdout, k-fold
Selection + regularization	Select variables or shrink coefficients	Best subset, ridge, lasso
CLASSIFY & DECIDE		
Logistic	Predict a yes/no outcome	Logistic regression
Thresholds	Turn a probability into a decision	ROC, cutoff
FLEXIBLE & UNSUPERVISED		
Trees	Split the predictors in sequence	Decision trees
Forests / boosting	Combine many trees	Random forest, boosting
PCA	Fewer combinations of the predictors	Principal components
Clustering	Group similar observations	K-means

Can we predict a new customer's credit Balance?

The bank has already given these customers a card and a **limit** — the most they may charge. They spend. Pay the bill in full, and nothing is left. Pay only part and the unpaid part stays on the card. That unpaid amount is the **balance**, in dollars. The bank charges interest on it. A balance close to the limit means the customer is using almost all of the credit they were given.

We have 400 of these customers. The file has Income, Limit, Rating, number of cards, and a few demographics. We want to predict **Balance**.

- **Problem:** How to select the variables that best help us to predict the credit card balance for a given client?
- $Y = \text{Balance}$: that unpaid amount, in **dollars**. Mean ≈ 520 , SD ≈ 460 .
- One row = one customer ($n = 400$).
- 11 predictors



Why not just select the variables we think are the best predictors? Supervise vs unsupervised selection?

Codebook for the Credit card dataset

Variable	Type	Unit	Description
Balance	numeric	dollars	Average credit-card balance (the Y)
Income	numeric	\$000s	Income
Limit	integer	dollars	Credit limit
Rating	integer	points	Credit rating
Cards	integer	count	Number of credit cards
Age	integer	years	Age
Education	integer	years	Years of education
Own	factor	—	Home owner (Yes / No)
Student	factor	—	Student (Yes / No)
Married	factor	—	Married (Yes / No)
Region	factor	—	East / South / West

Based on the codebook, what other questions could be of great relevance for the company?

What we will do

- 1 How can one select a good predictive model among many? How can one avoid overfitting?

- 2 Model Selection methods: Best subset and stepwise

- 3 How to select the best models among good options. Important statistics: C_p , AIC, BIC, and adjusted R^2
 - Indirectly: adjust the training error with those four statistics.
 - Directly: estimate the test error with cross-validation.

- 4 Limit $\approx$ Rating: why hard include/exclude is awkward

- 5 Ridge and lasso: shrink with λ ; lasso can set some slopes to zero

- 6 Holdout on this file: lasso ties OLS on this split; ridge loses by a few dollars — one split is not a ranking

How can one select a good predictive model for balance among many?

Let's say we start with the model on all 11 predictors on this file ($n = 400$, $p = 11$). We typically fit the model using least squares (LS).

$$Y = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p + \epsilon$$

How the model can be improved?

How can one select a good predictive model among many? How can one avoid overfitting?

TWO reasons to do something else:

Prediction accuracy

when columns are many or move together, models can include variables that add a lot of noise

- If relationship between variables is linear, β estimates will have low bias.
- If $n \gg p$, β estimates will also have low variance. However,
- If n is not much larger than p , there will be a lot of variability $\rightarrow$ overfitting

Model interpretability

some columns add almost nothing

- Unnecessary complexity $\rightarrow$ model less easy to interpret
- If p is comparable to n , LS variance explodes. Overfitting!

Last week, training MSE could be less informative for predictions outside the sample. Will training RSS (MSE) would still be not useful when we choose which columns to keep before running the CV?

Three sets of methods

1

Subset selection methods

Keep the predictors related to the response.

Best subset, forward, and backward.

2

Shrinkage methods

- Use all p predictors.
- Pull coefficients toward zero.
- Cuts variance vs. least squares.

Ridge and lasso.

3

Dimension reduction methods

- Compress the columns (PCA), then model.
- Later in the course — not today.

In this session, we work the first two set of methods to better model credit card balance. Dimension reduction (PCA) is later in the semester

If Limit and Rating move almost as one, which method keeps both while reducing the problem, instead of removing Rating from the model?

Best subset: for each size, keep the best RSS model

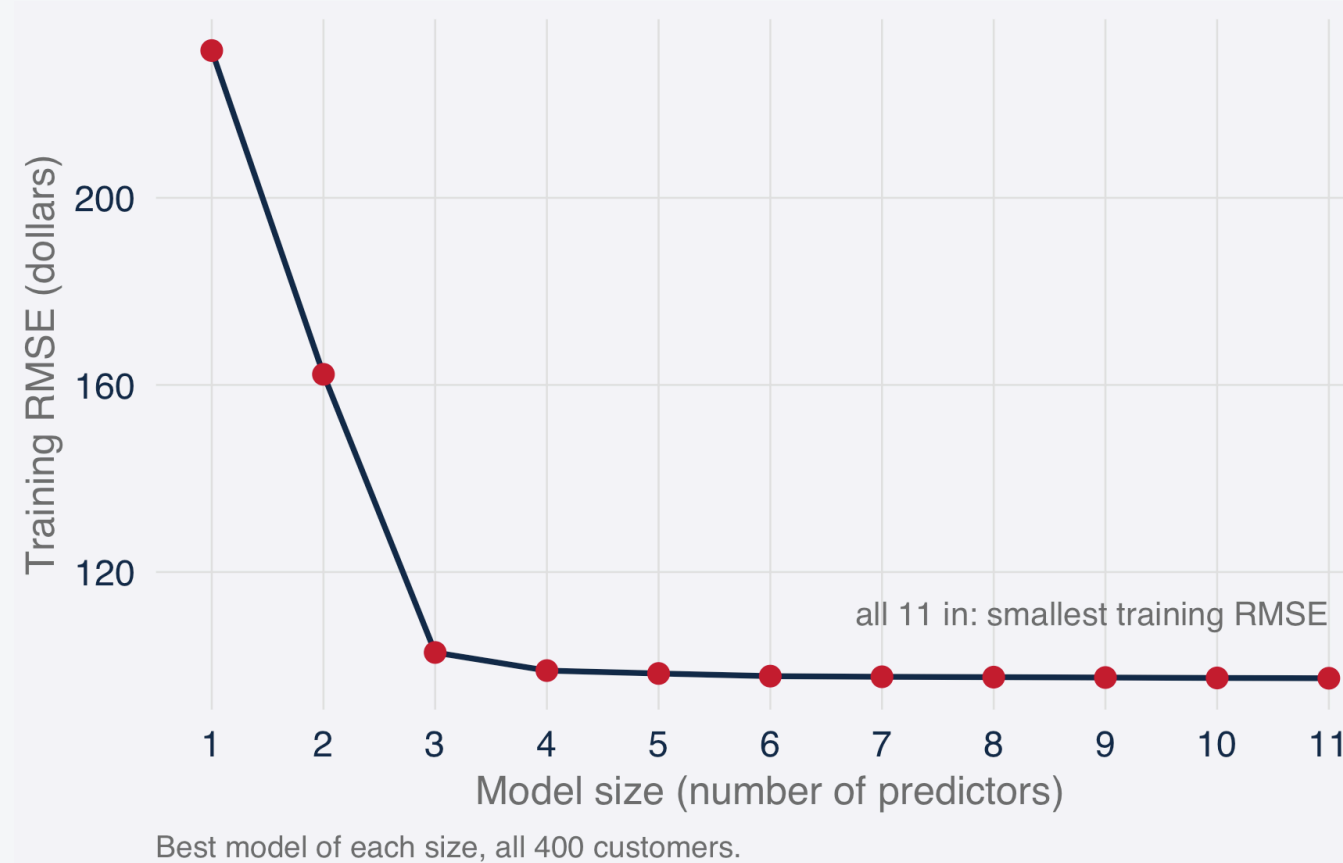
- For $k = 1, 2, \dots, p$: fit every model with exactly k predictors; keep the one with the smallest RSS (or RMSE).
- With $p = 11$, we are running $2^{11} = 2048$ models

$$\hat{M}_k = \arg \min_{|M|=k} \text{RSS}(M)$$

M = a subset of predictors; RSS = sum of squared Balance leftovers, in dollars², on the customers we use to fit the model.

```
1 library(leaps)
2 fit <- regsubsets(Balance ~ ., credit, nvmax = 11)
```

Example: Credit — training RMSE of the best model of each size.



How to pick among the best models of each size?

Options? when to use each one?

Stepwise selection

Limitations of best subset selection

- It can be **computationally unfeasible**: 2^p fits. Here $p = 11$, so 2,047 models; with $p = 20$ about a million.
- A **larger search** can favor a model that fits the training data and predicts poorly on new data. The search itself adds variance.

Solution: stepwise methods

```
1 regsubsets(Balance ~ ., credit, nvmax = 11, method = "forward")
2 regsubsets(Balance ~ ., credit, nvmax = 11, method = "backward")
```

Forward stepwise selection

- Start with a model containing no predictors.
- Add predictors one at a time, until all are in.
- At each step, add the one that gives the greatest additional improvement.

Backward stepwise selection

- Start with a model containing all predictors.
- Remove the least useful predictor, one at a time.

Both searches are greedy. Which one cannot even start when $p > n$?

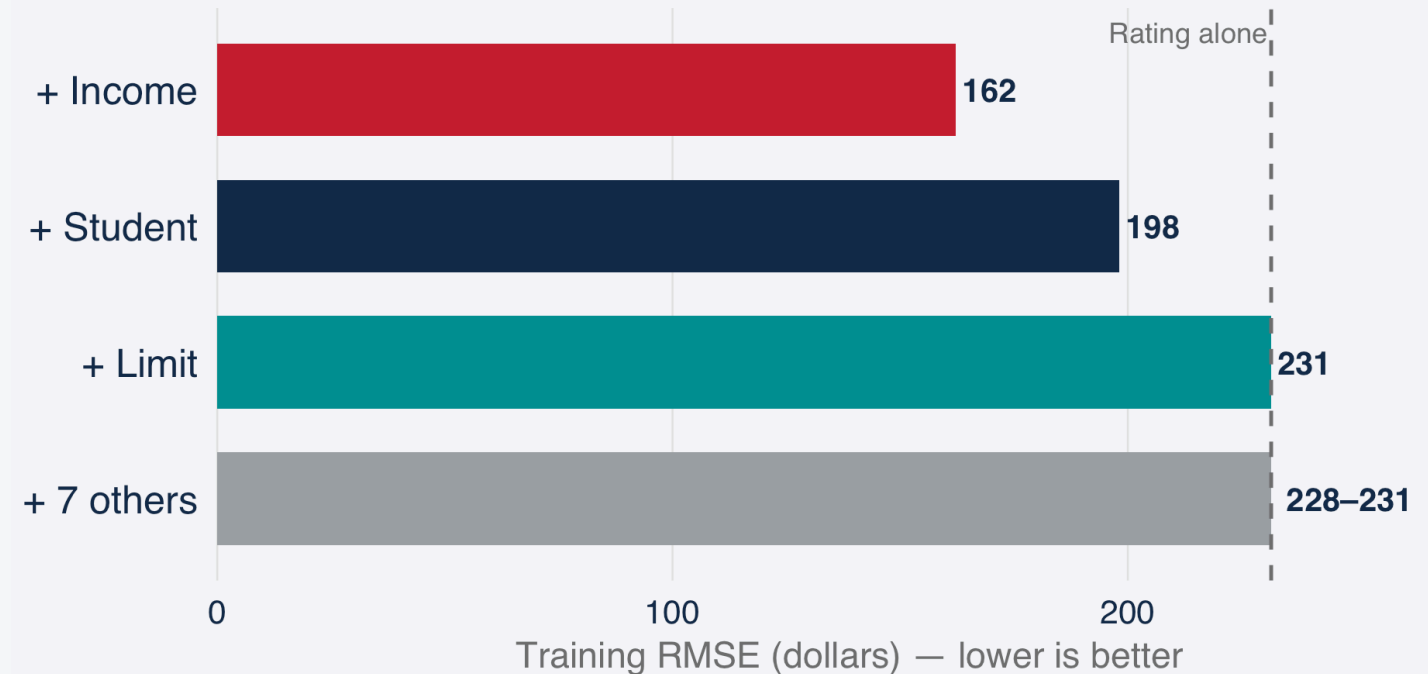
Forward stepwise selection: greedy from the bottom

1. Let M_0 be the null model: intercept only.
2. For $k = 0, 1, \dots, p - 1$: try every model that adds **one** predictor to M_k ; keep the addition with the smallest RSS. Call it M_{k+1} .
3. Pick **one** of $M_0, \dots, M_p$ using Cross-validated prediction error, Cp (AIC), BIC, or adjusted R^2 .

```
1 regsubsets(Balance ~ ., credit, nvmax = 11,  
2           method = "forward")
```

Example: Credit — step 2. Every bar already includes Rating. $M_2 = \text{Rating} + \text{Income}$

Dashed = Rating alone. Teal = Limit now does nothing. Income + Limit (165) is not a candidate.



Best subset at size 2 looks at all 55 pairs. Forward looks at the 10 that contain M_1 . Why can it miss the best pair?

More on forward stepwise selection

- Forward stepwise has a clear **computational advantage** over best subset.
- However, it is **not guaranteed** to find the best possible model among all 2^p models. Why?

A case forward cannot see

- Suppose the best one-variable model contains X_1 , and the best two-variable model contains X_2 and X_3 — and **not** X_1 .
- Forward is stuck: after picking X_1 it can only produce $\{X_1, \text{something}\}$. It will never see $\{X_2, X_3\}$.

On Credit. The one-predictor model on the last slide is Rating. That is the X_1 in the box. The t searches first disagree at **4 predictors**.

Best subset's 4-predictor model

Income Limit Cards Student ~~Rating~~

Rating is **out**. Best subset was allowed to drop M_1 . That is the analog of $\{X_2, X_3\}$.

Forward's 4-predictor model

Income Limit **Rating** Student

Rating is still **in**. Forward cannot drop M_1 , so it can never produce the set above.

Is backward stepwise immune to this?

No. Backward can drop a column that is useless alone but useful together with another column it already dropped. Greedy both ways.

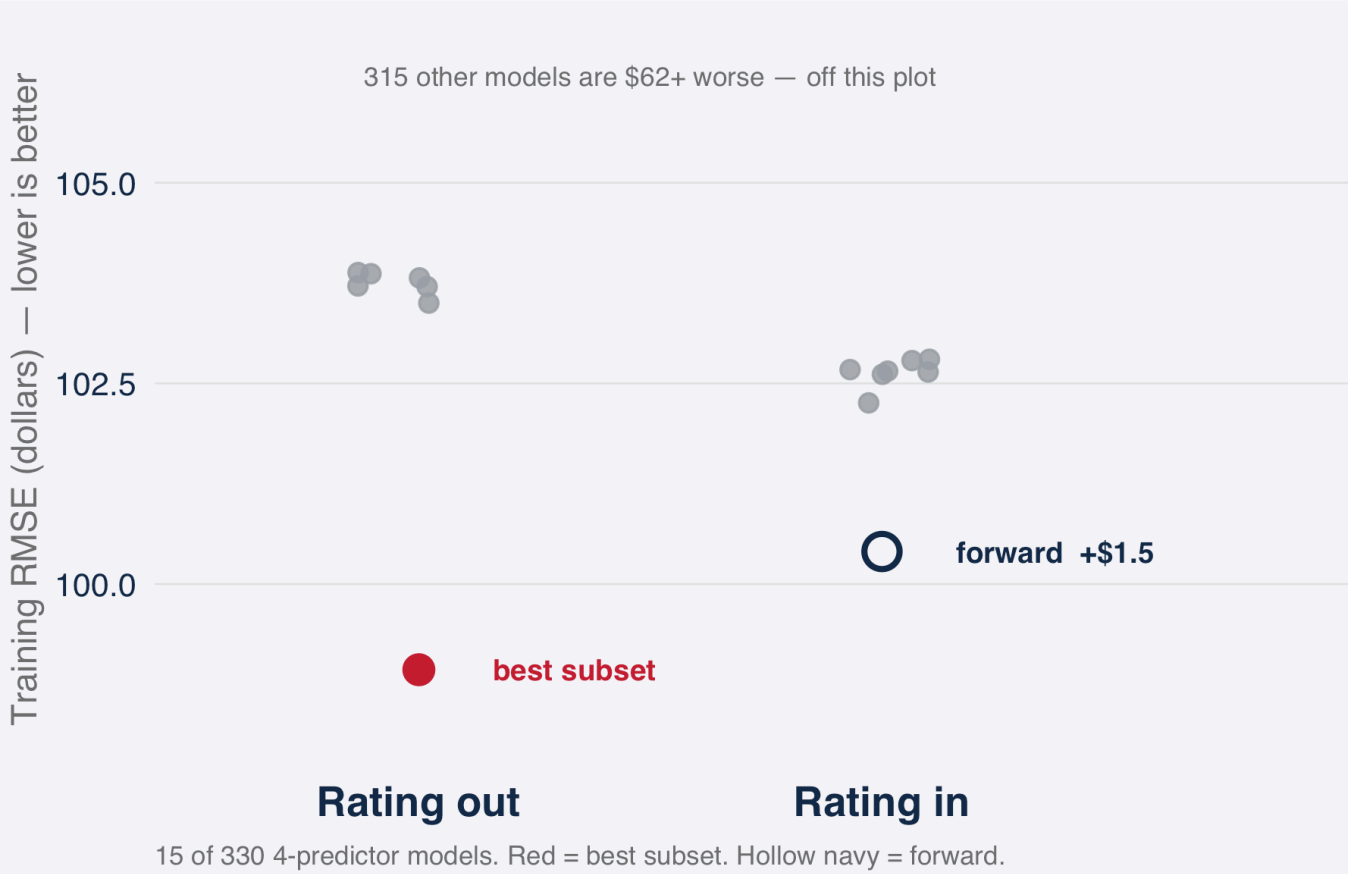
Example: Modelling Credit balance. Best subset vs. forward stepwise

First 5 sizes on this file ($n = 400$). **Red** = the two searches disagree.

Size	Best subset	Forward stepwise	Same model?
1	Rating	Rating	yes
2	Income, Rating	Income, Rating	yes
3	Income, Rating, Student	Income, Rating, Student	yes
4	Income, Limit, Cards, Student	Income, Limit, Rating, Student	no
5	Income, Limit, Rating, Cards, Student	Income, Limit, Rating, Cards, Student	yes

- Sizes 1–3 agree. At size 4 the best model has **Cards** and not **Rating**. Forward cannot drop its early pick.
- At size 4, training RMSE is 1.5 dollars higher than best subset. Same customers, same Y .

Every 4-predictor model on this file (330 models). The best dropped Rating. Forward kept it and came second.



Forward is second, by about \$1.50. Is that difference small enough to rely on forward stepwise when there are more predictors?

Backward stepwise selection: greedy from the top

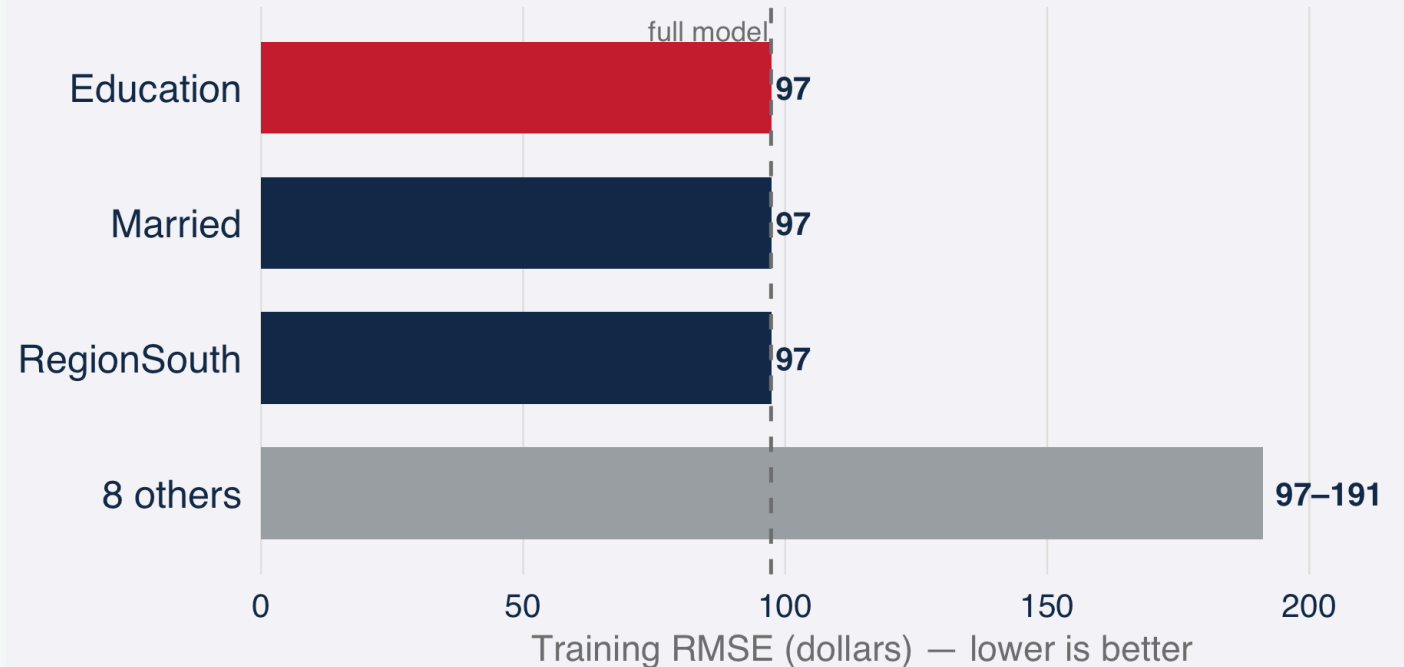
1. Start with the full model M_p : all 11 predictors.
2. For $k = p, p - 1, \dots, 1$: try every model that drops **one** predictor from M_k ; keep the drop that raises RSS least. Call it M_{k-1} .
3. Choose one of $M_0, \dots, M_p$ with a criterion or CV.

Limitations

- Not possible when $p > n$: the full model cannot be fit.
- When p is large, use **forward** stepwise selection (or shrinkage — coming).

Example: Credit — first drop from the full model. Each bar is training RMSE after that one drop. **First to leave: Education**

Red = first to leave. Dashed = full model.



Forward started with **Rating**. Backward ends with **Limit**. Same customers, opposite start.

If $p = 500$ and $n = 300$, which search can you still run?

Choosing the optimal model

- Subset selection gives us one model per size. We still have to choose the size.
- We want the size with the **lowest test error**.
- Training RMSE falls as we add predictors (slide 10), so R^2 and RSS on the fitted rows **cannot** choose it. The two strategies below estimate the test error instead.

Strategy 1 — indirectly

- Estimate the test error: adjust the training error to account for the bias due to overfitting.
- Four options: Mallows' C_p , AIC, BIC, adjusted R^2 .

Strategy 2 — directly

- Estimate the test error with a validation set or cross-validation.
- **We have done this before** (Session 03).

Strategy 1 adjusts the training error. Strategy 2 scores rows that were not used to fit the model. What does each strategy require?

Strategy 1: indirectly estimating the test error

- Training RSS falls as we add predictors, so it cannot choose the size.
- Mallows' C_p , the Akaike information criterion (AIC), and the Bayesian information criterion (BIC) keep that fit and add a penalty for each predictor. A larger penalty favors a smaller model. Adjusted R^2 corrects R^2 for the number of predictors; it does not add a penalty to RSS.

Mallows' C_p , AIC, and BIC = **RSS + a penalty for d**

Mallows' C_p and AIC use a penalty of about **2** per predictor. BIC uses $\log(n) = \mathbf{6.0}$ on these 400 customers, so it prefers smaller models.

Mallows' C_p

$$C_p = \frac{1}{n} \left(\text{RSS} + \mathbf{2d\hat{\sigma}^2} \right)$$

Akaike information criterion (AIC)

$$\text{AIC} = -2 \log L + \mathbf{2d} \approx C_p \text{ for least squares}$$

Bayesian information criterion (BIC)

$$\text{BIC} = \frac{1}{n} \left(\text{RSS} + \mathbf{\log(n)d\hat{\sigma}^2} \right)$$

Adjusted R^2

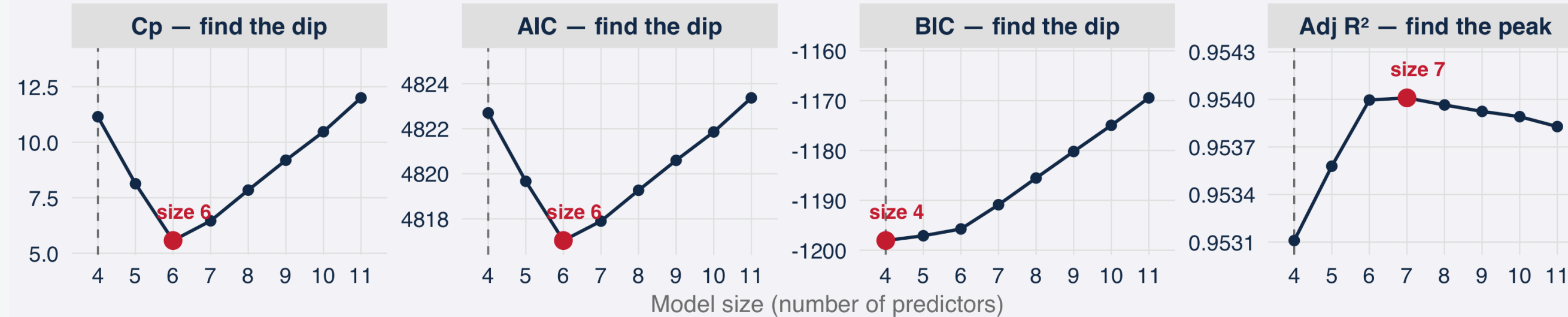
$$\text{Adjusted } R^2 = 1 - \frac{\text{RSS}/(n - d - 1)}{\text{TSS}/(n - 1)}$$

n = customers; d = predictors in the model; RSS, TSS in Balance dollars²; $\hat{\sigma}^2$ from the full least-squares fit; L = maximized likelihood. The **red** term is the penalty for size.

Example: Credit. C_p , AIC, BIC, and adj R^2 do not pick the same size

Best subset's best model of each size from slide 9, scored four ways. Where does each score bottom out (or peak)?

Same four statistics, from size 4 on. Sizes 1–3 are off this plot, so the minimum and the peak can be read.



- Mallows' C_p and AIC select size **6**. BIC selects size **4**. Adjusted R^2 selects size **7**.

BIC selects size 4. Mallows' C_p selects size 6. Which two predictors does the smaller penalty include?

The smaller penalty keeps **Rating and Age**. BIC leaves them out.

Strategy 2: directly estimating the test error

- Use validation or cross-validation, as in Session 03. For each candidate model, estimate the error on rows that were not used to fit it.
- **Advantages:**
 - It estimates the test error directly.
 - It does not require an estimate of the error variance, and it does not assume the errors are normal. Prediction error can be measured by RMSE.
- **Disadvantages:**
 - Each candidate model is refit on every fold, so the computation is heavier.
 - With a small number of predictors, as on this file and the computational capacity of modern laptops, that's not longer a problem in most cases.
- **Hold out part of the sample.** Split once: 280 customers for fitting and choosing, 120 held out. Cross-validation uses the 280. The 120 are used once, after the model is chosen. Earlier subset results used all 400 customers.

1	2	3	4	5	6	7	8	9	10	HELD OUT
---	---	---	---	---	---	---	---	---	----	----------

Training rows (n = 280) — ten folds: choose the model size or λ (for regularization later) here

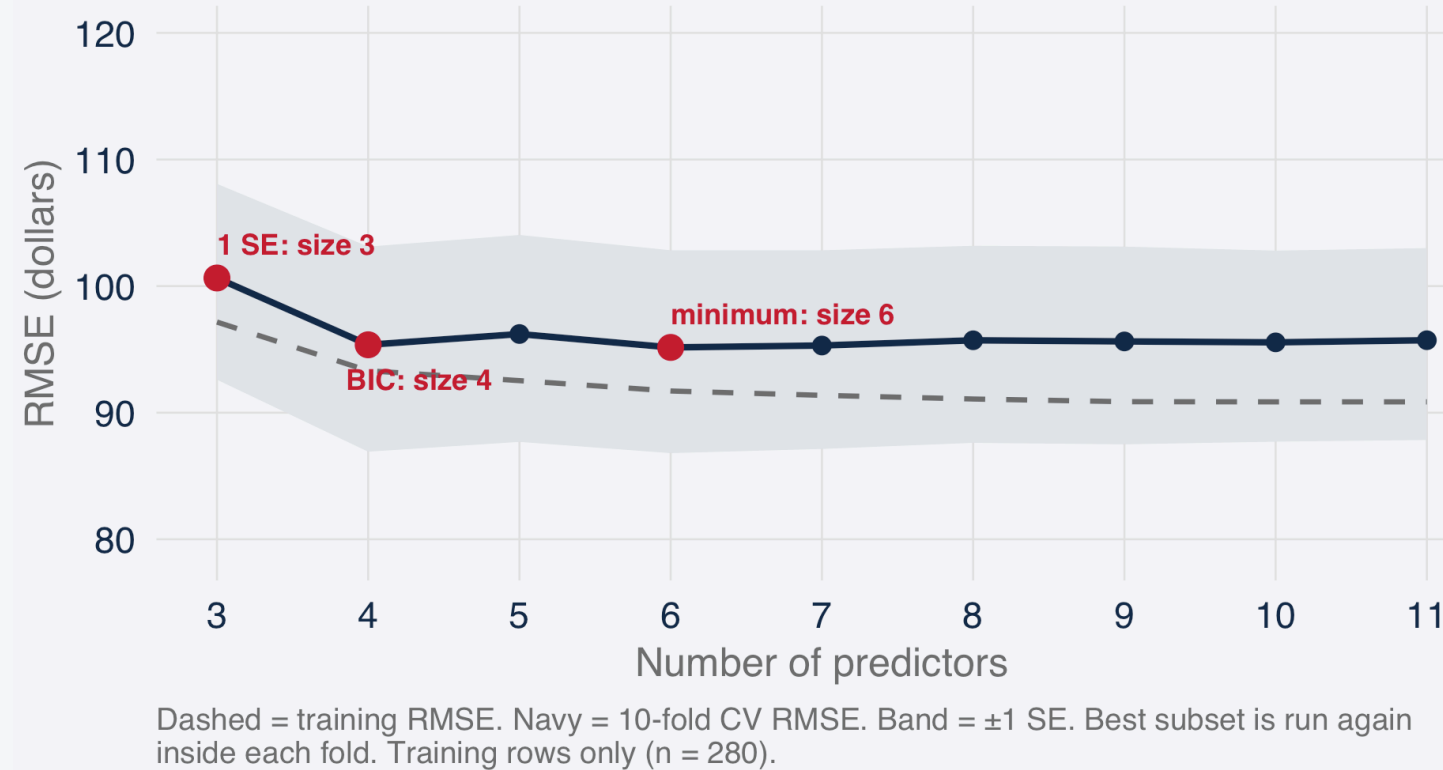
Held-out rows (n = 120) — used once, after the model is chosen

In each fold, best subset is run again on nine folds. The tenth fold is used only to compute the error.

Example: Credit. CV error by model size, on the training rows

- **Dashed line:** training error. It keeps falling, so it cannot choose a size.
- **Navy line:** 10-fold cross-validation error on the 280 customers. Best subset is run again inside each fold. The line flattens.
- **Minimum,** the red mark at size **6**: about \$95.
- **One standard error,** size **3**: about \$101. The smallest size these folds cannot separate from the minimum.
- **BIC** on these 280 customers selects size **4**. It sits on the flat part of the navy line, between size 3 and size 6.

Training error keeps falling. Cross-validation error flattens. Sizes 1 and 2 are off this plot.



Training R^2 cannot choose the size either. It rises every time a predictor is added.

Size 4 and size 6 have almost the same cross-validation error, about \$95. Average Balance is about \$520. Size 6 uses more predictors. Which size do you use, and what are you giving up?

Why not choose the predictors by their p-values?

Size 4 and size 6 predict about equally well. A colleague may say: keep the predictors with small p-values and drop the rest. Limit and Rating show why that is a bad rule for this decision.

- Limit and Rating move together. Both describe how much credit the customer has.
- Fit alone, each looks important.
- Fit together, one of them looks unnecessary.
- Put every predictor in the model, and the result changes again.

Credit. Same customers. The p-value depends on which other predictors are in the model.

Variable	Alone (1 predictor)	With its twin (2 predictors)	Full fit (11 predictors)
Limit	3×10^{-119}	0.70	1×10^{-8}
Rating	2×10^{-120}	0.02	0.02

The cross-validation error did not tell you to drop Rating or Limit. The p-value should not either. Keep both. Next we shrink the slopes.

Model selection: what to remember, and what to do

What to remember

- A better fit on the cases used to build the model is not a better prediction. Adding a predictor improves that fit.
- A search that cannot undo an early choice can miss a better model.
- Criteria that penalize model size differently will not always select the same model.
- A p-value depends on which other predictors are in the model. It is not a prediction error.

What to do

- Split the sample before you choose. Choose on one part. Report the error on the other part only after the model is fixed.
- If you cross-validate a search, run the search again inside each fold.
- When two models have almost the same prediction error, use the one with fewer predictors.
- When two predictors move together, do not drop one because of a p-value. Shrink both. This is what we will do next in model regularization!

Regularization: ridge and lasso

Three classes of methods — where are we?

1. Subset selection — done

Identify a subset of the p predictors related to the response. Best subset, forward, backward. A column is in or out.

2. Shrinkage — now

Fit a model with **all** p predictors. Estimated coefficients are **shrunk** toward zero relative to least squares.

Effect: it reduces variance!

Options: ridge regression, lasso.

3. Dimension reduction — later

Compress the columns (PCA). Later in this course, not today.

- In subset selection, a predictor is included or it is left out. In ridge and lasso, every predictor stays in the model, and the coefficients are shrunk toward zero. The choice is no longer how many predictors to include. The choice is λ , which controls how far the coefficients are shrunk.

Limit and Rating move together. Should the method drop one of them, or keep both and shrink their coefficients?

Shrinkage methods: ridge regression and lasso

- The subset selection methods use least squares to fit a linear model that contains a **subset** of the predictors.
- As an alternative, we can fit a model containing **all** p predictors using a technique that **constrains** or **regularizes** the coefficient estimates — equivalently, that **shrinks** them toward zero.
- It may not be obvious why such a constraint should improve the fit. It turns out that shrinking the coefficient estimates can **significantly reduce their variance**.

Least squares

Same coefficient, estimated on many samples. Centered on the least-squares value. Spread out.



After shrinkage

Same samples. Closer to each other, and closer to zero. A little bias, less movement.



Not estimates from this file. Each dot is one sample. λ moves the cloud toward zero and makes it tighter. That is the variance reduction above.

If the goal is to predict Balance for a customer not used to fit the model, do we need the coefficient to be unbiased, or do we need it to stay stable when the sample changes?

Ridge regression

$$\text{RSS} = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \quad \longrightarrow \quad \text{least squares picks the } \beta \text{ that make this smallest.}$$

$$\hat{\beta}^{\text{ridge}} = \arg \min_{\beta} \left\{ \text{RSS} + \lambda \sum_{j=1}^p \beta_j^2 \right\} \quad \text{penalty}$$

y_i is customer i 's Balance, in dollars. x_{ij} is a predictor, such as Income, Limit, or Rating. The fit uses the 280 training customers.

Ridge estimates the coefficients as the values of β that minimize the residual sum of squares plus the penalty in red. The penalty is λ multiplied by the sum of the squared slopes. Because each slope is squared, a slope of 2 contributes four times as much as a slope of 1. A larger λ multiplies that sum, and the resulting slopes are smaller than the least-squares slopes.

- If $\lambda = 0$, ridge is least squares.
- As λ grows without limit, every slope goes to zero. The intercept alone predicts the mean Balance.
- λ is not estimated with the slopes. We choose it by cross-validation.
- The intercept is not penalized.

If we chose λ to make the residual sum of squares on the 280 training customers as small as possible, what value would we get, and why do we use cross-validation instead?

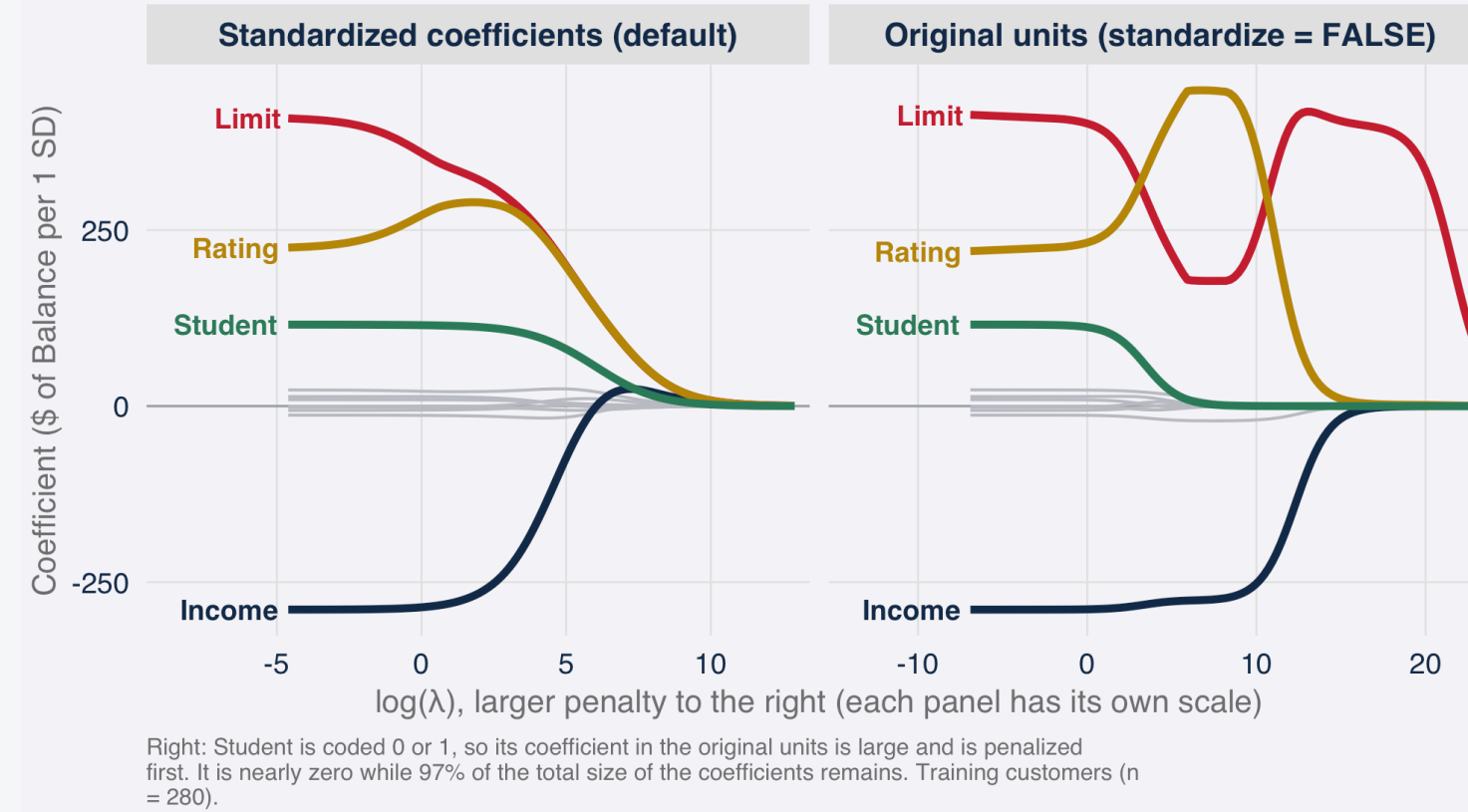
Why standardize the predictors before ridge?

- The penalty adds the squared coefficients. Limit is in dollars and Student is coded 0 or 1, so those coefficients are not comparable.
- On the left, the predictors were standardized. As the penalty increases, the coefficients move toward zero together.
- On the right, the predictors stay in the original units. Student is reduced first, because a 0 or 1 predictor has a large coefficient.

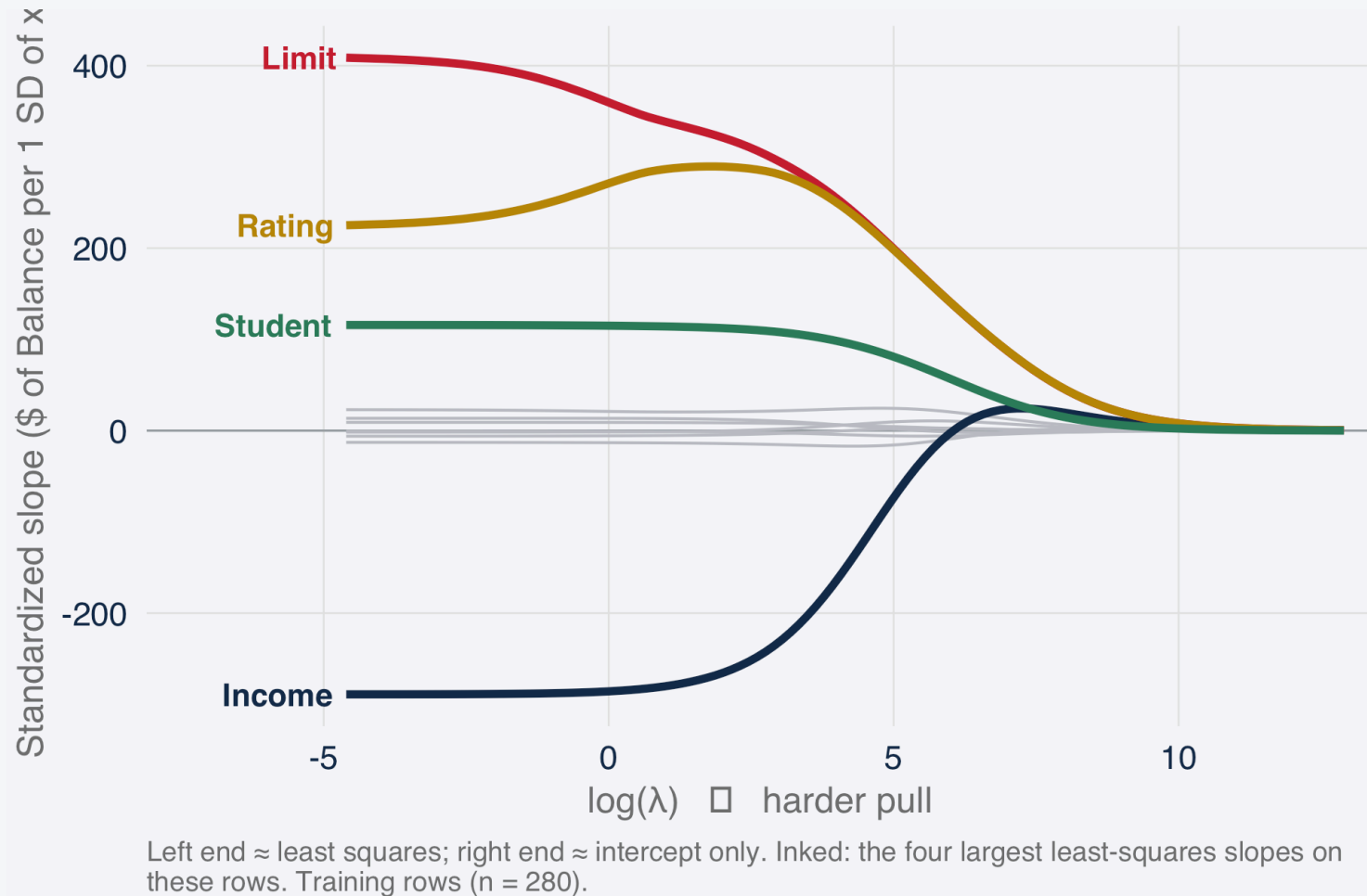
```
1 ridge <- glmnet(x_train, y_train, alpha = 0)
2 # standardize = TRUE is the default
```

This default is the left panel.

Example: Credit. Larger penalty to the right. Left: predictors standardized before the penalty. Right: original units. Both vertical axes are dollars of Balance per 1 standard deviation.



Application to the Credit data: the ridge path



Observations:

1. First, we **standardize** the data — otherwise this picture is unreadable.
2. On the Y axis: the ridge coefficients, one curve per column for different values of λ . Left = least squares, right = heavy shrink.
3. As λ increases the penalty is higher, so coefficients **shrink toward zero — but they are not exactly zero.**
4. **Four variables** stay large the longest: **Limit, Income, Rating, Student** — the same group of variables as in the best subset.

The previous slide showed that the picture is misleading when the predictors are left in their original units. What did standardization change about the way we read Limit and Student on this figure?

Why does ridge improve over least squares?

Test error is squared bias, plus variance, plus an error that remains even with the true model.

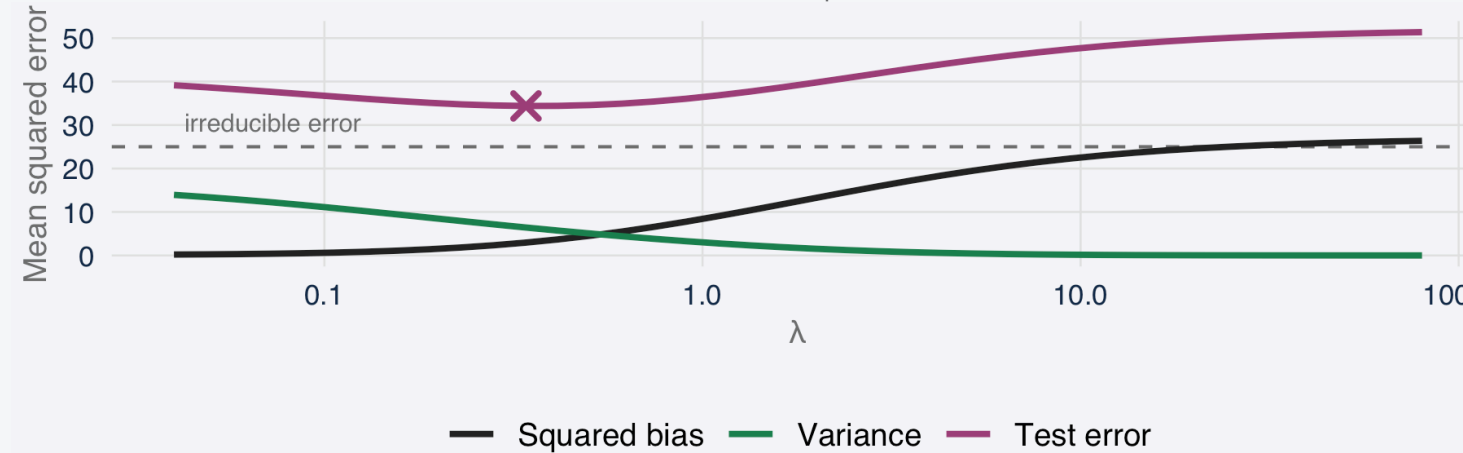
$$\text{test MSE} = \text{Bias}^2 + \text{Variance} + \text{irreducible error}$$

In the simulation, variance falls and squared bias rises. Test error falls, then rises. The mark is the minimum.

On Credit, cross-validation estimates that test error. It stays near 95–96 dollars, then rises.

Across 200 resamples, Limit and Rating vary by about 87 dollars at least squares, and by less than 7 dollars when λ is about 50.

A simulation with a known truth. 100 observations and 40 predictors. Not the Credit file.



Credit. This estimates the test-error curve above, on the 280 training customers.



The lasso

Ridge keeps all 11 predictors. On the ridge path, the coefficients moved toward zero, and none of them reached zero.

$$\hat{\beta}^{\text{lasso}} = \arg \min_{\beta} \left\{ \text{RSS} + \lambda \sum_{j=1}^p |\beta_j| \right\}$$

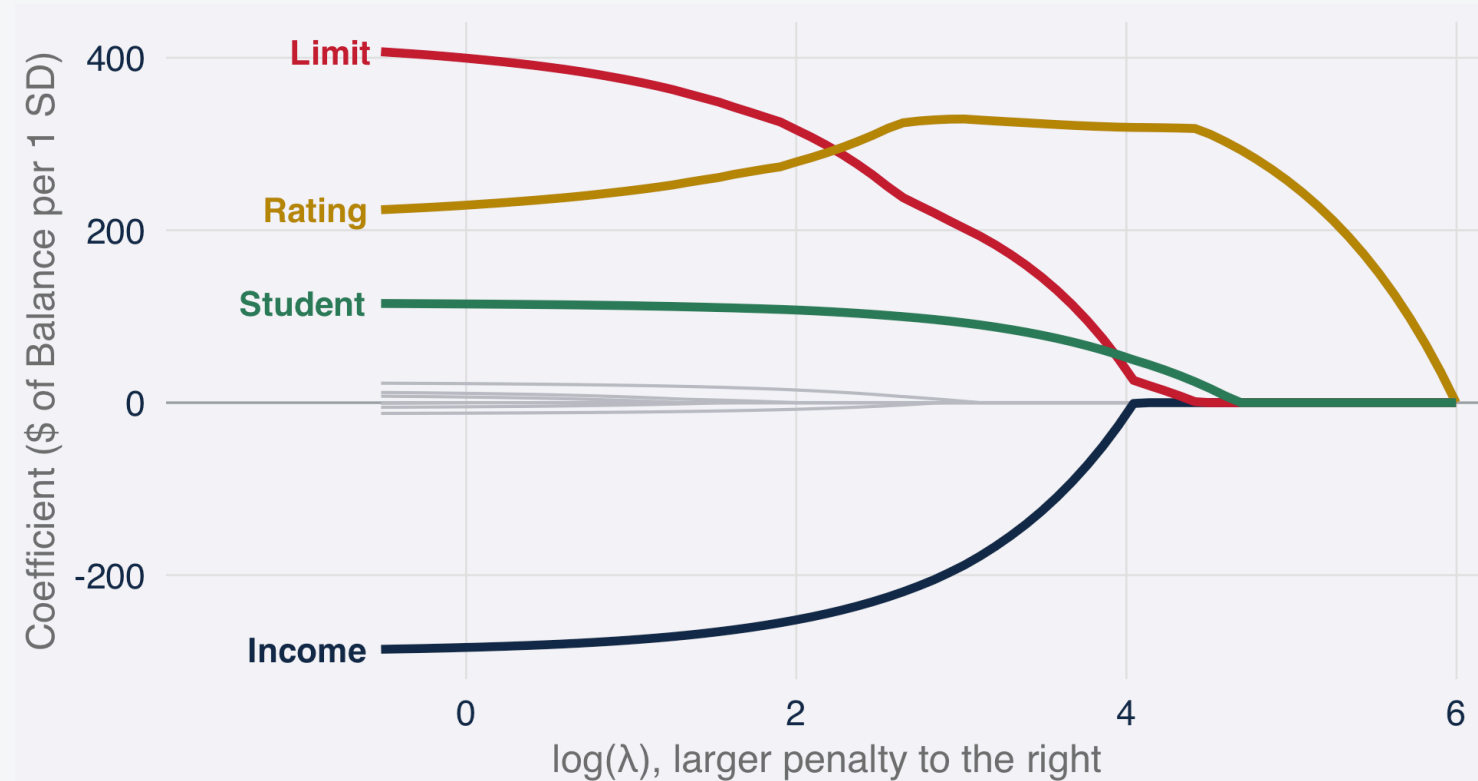
The residual sum of squares is the same as in ridge. The red term is λ times the sum of the absolute values of the coefficients. The fit uses the 280 training customers, after the predictors are standardized.

- A large enough λ sets some coefficients exactly to zero, and those coefficients stay at zero. A coefficient of zero means that predictor is left out of the prediction.
- The intercept is not penalized. λ is chosen by cross-validation, as with ridge.

Ridge used the square of each coefficient, and none of them reached zero. The red term uses the absolute value. What can this penalty do that the squared penalty did not?

The lasso path on Credit

Same 280 training customers and the same standardized predictors as the ridge path. Only the penalty changed: the sum of absolute values.



- As λ increases, coefficients reach exactly zero and stay at zero. Own reaches zero first. Rating is last.
- On the ridge path, the same coefficients only approached zero. None of them was exactly zero.
- Limit and Rating move together. The lasso can set one of those coefficients to zero and keep the other. Both predictors are still associated with Balance.

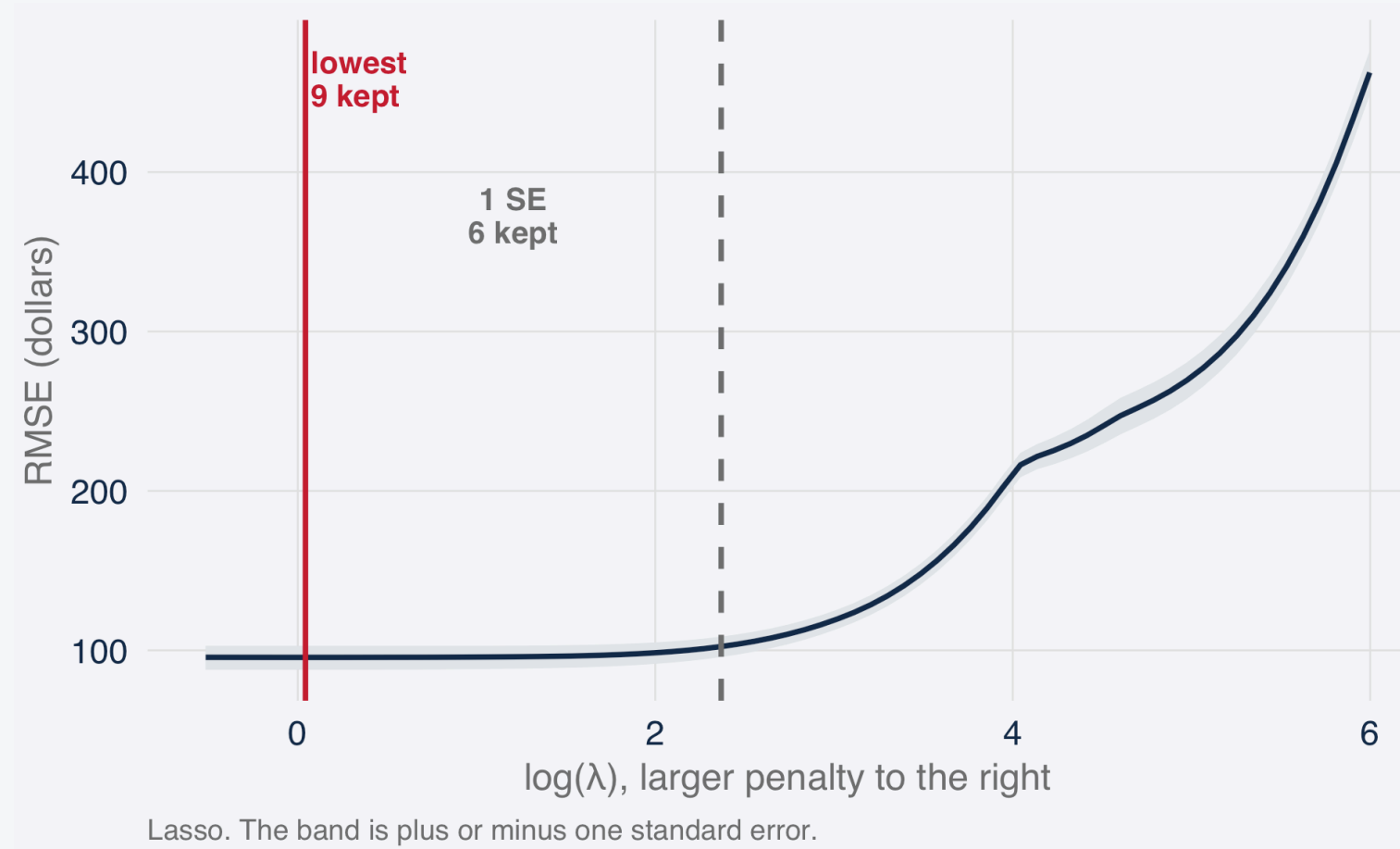
```
1 lasso <- glmnet(x_train, y_train, alpha = 1)
```

A coefficient that reaches zero on this path: does that mean the predictor has no association with Balance?

How cross-validation chooses λ

The curve is the estimated test error at each λ . It uses the 280 training customers. A larger penalty is to the right.

- Training error is smallest at $\lambda = 0$. That does not choose λ .
- The red line is the lowest point on the curve. About 9 coefficients are kept.
- The dashed line is the largest λ within one standard error of that point. About 6 coefficients are kept. The error is almost the same.



The two lines have almost the same error. The dashed line keeps fewer coefficients. Which λ would you use?

Does shrinkage beat least squares on new customers?

Prediction error on the 120 held-out customers. They were not used to fit or to choose.

- Least squares: about 114 dollars.
- Lasso: about 113–114.
- Ridge, default grid: about 121–125.

Lasso matches least squares. Default ridge is higher, by about 7–11 dollars.

Ridge's cross-validation error was about as low as the lasso's. Why is ridge higher on this graph?

The default ridge grid stops near $\lambda = 40$, and the minimum is on that edge. A wider grid gives about 113–114 dollars.



The axis runs from 90 to 140. Predicting the training mean is about 459 dollars, off this plot.

What to remember

- 1. A lower training error is not a better prediction.
- 2. Cross-validation chooses the number of predictors, or λ .
- 3. If those errors are almost the same, take the shorter model.
- 4. If two predictors move together, do not drop one for a p-value. Ridge keeps both. A lasso zero is not “no association.”

On Credit.

Question	Answer
Cp, AIC, BIC, adjusted R ²	Cp 6, AIC 6, BIC 4, adjusted R ² 7
Cross-validation	minimum size 6; 1 SE size 3
Lasso	9 coefficients at the minimum, 6 at 1 SE
Held-out customers, once	about 114 dollars. Least squares, lasso, and ridge agree

How do we select a good predictive model among many?